

Lecture 3

Packaging

oliverni

(slide credits dkessler, mdcha)

Administrivia

- Vitamin 2 and Lab 2 were due last week
  - Fill out the Late Lab Form if needed.
- Lab 3 due this week
- Use your resources!
  - Your facilitators!
  - Ed, Gradescope
  - OCF Slack (ocf.io/slack) or Discord (ocf.io/discord)
  - google.com

Topics

- Linux distributions
- Package managers
- Packaging for Debian



Linux Distributions and Debian

Distributions?

- Linux = kernel
  - Manages resources & allocates to programs
- What about everything else in the OS?
  - Programs: bash, cat, ls, etc.
  - Init system
  - Package manager
  - Make it actually usable

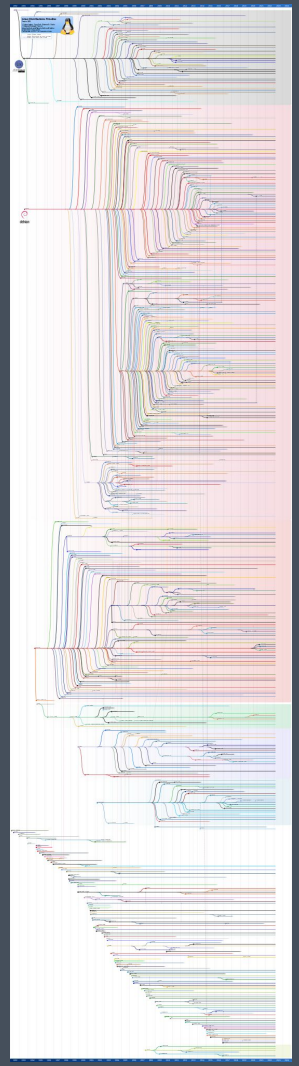
Distributions?

- Linux kernel + other software = operating system
  - What other software to choose?
  - How to manage this software?
  - How should using the system work for users?
- Different Linux *distributions* make different choices about everything besides the Kernel

Distro Chart

- Distributions are spun off a lot
  - Debian -> Ubuntu, RHEL -> Scientific Linux, Arch -> Manjaro

https://upload.wikimedia.org/wikipedia/commons/1/1b/Linux_Distribution_Timeline.svg



Installing software: package managers

How do you install software?

- What files do you need? How do you get them?
- How much work does the end user need to do?
- How much work does the developer need to do?
- Anyone else involved in distributing software?
- How do distribute updates?
- Any security concerns?

Naïve solution

- Download a set of files
  - compiled binaries,
  - documentation,
  - Config files
  - license(s)
  - Internationalization / localization files
- Any problems?

Naïve solution and its issues

- Which files do I need to download?
- Where do I put each file?
- Configuration issues?
- Will it work on my system?
- Updates?

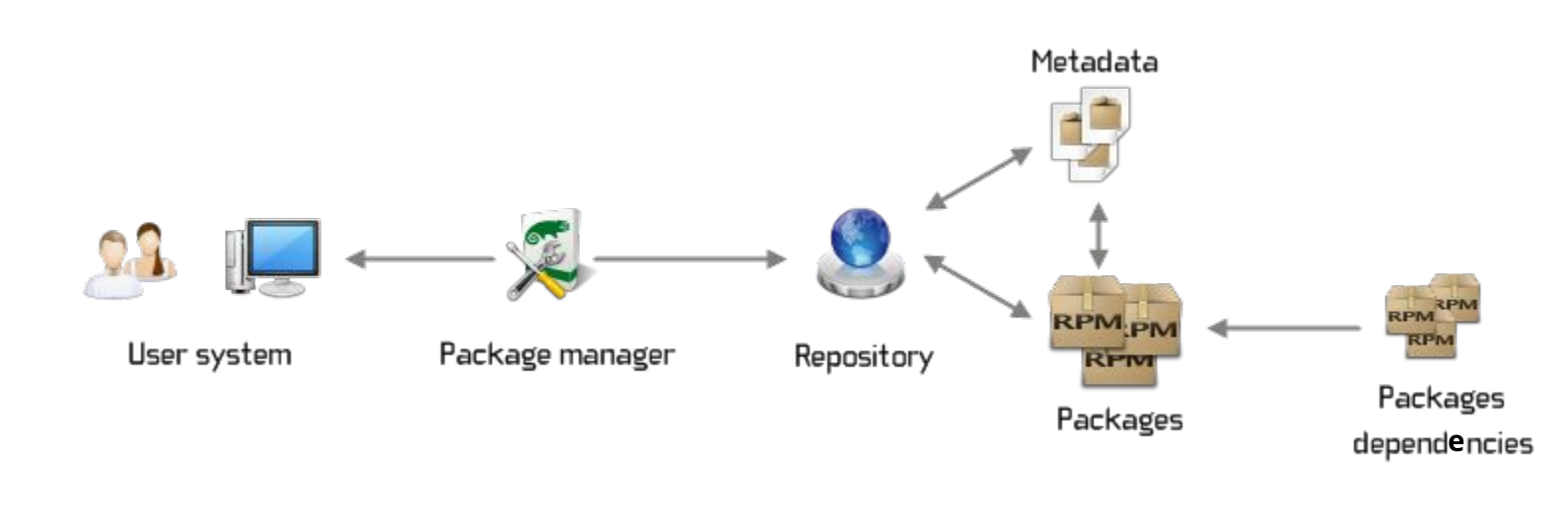
Tarball solution

- Download a tarball from the application developer (“upstream”):
  - Should the source code be compiled or pre-compiled?
  - What if the software depends on other software being installed?
  - Each Linux distribution is different: does the application developer need to learn the quirks of each?
  - How do we get updates?
  - How can users trust that updates won’t break or be disruptive?

Packaging!

- A centralized system for managing package metadata
  - Each Linux distribution maintains repositories that contain lists of packages available to download
  - Volunteers (package maintainers), on behalf of distributions, are responsible for packaging developers' software
  - Each distribution has processes and policies for how software should be packaged

Packaging!


Metadata
User system
Package manager
Repository
RPM
Packages
Packages dependencies

What about App Stores?

Linux distribution package repositories and app stores are both are centralized places to get software,

...but there are key differences.

Packages vs Apps: distributing updates

Packages:

- Maintainers vet updates
- Disruptive updates could get delayed
- Users who want to test newer versions can subscribe to newer tracks (Debian: “testing” or “unstable”)

Apps

- Developers push updates directly to users
- Canarying: developers might randomly select users to test new versions on
- Minimal bureaucracy/friction to pushing updates

Packages vs Apps: security & trust

Packages

- Users trust distribution to not package sneaky software
- Package maintainers patch out anti-features when necessary
- Some distros will include AppArmor/SELinux profiles as an extra security precaution

Apps

- Users hope developers won't be sneaky
- Permissions, consent dialogs, and OS features (e.g. "x is reading your location") place technical limitations
- Strong sandboxing/isolation

Packages vs Apps: dependencies

Packages

- Packages can depend on other packages (libraries)
- Dependencies can get updated independently of packages that depend on them
- Your distribution ensures that dependency upgrades don’t break anything

Apps

- Apps are sandboxed and isolated, can only depend on the underlying OS
- App developers are responsible for keeping dependencies up to date
- Developers are responsible for testing when dependencies are updated

Debian & apt

Debian

- One Linux distribution, debuted in 1993
- Why Debian?
  - User-friendly: Works out-of-the-box, easy to install
  - Extremely stable: new release only every 2 yrs
  - Popular: used as base for Ubuntu, etc.
  - Cute naming scheme
- Used at the OCF (maybe not anymore soon....)

dpkg & apt

- Debian’s package management system
  - Packages end in `.deb`
- Backend: dpkg
  - Just installs a package
- Frontend: apt
  - Reads from package repositories
  - Handles dependency resolution
  - Dispatches to dpkg internally

apt commands

- apt update
  - Fetches latest lists of available packages from repositories
  - Doesn’t actually update any installed software!
- apt search <package name>
  - Searches for available packages in the catalog
- apt install <package name>
  - Installs packages from catalog
  - Also installs its dependencies!
- apt remove <package name>
  - Hopefully self-explanatory
- apt upgrade / apt dist-upgrade
  - Updates installed packages

A Demonstration of Package Installation

Install random package (cowsay).

What just happened

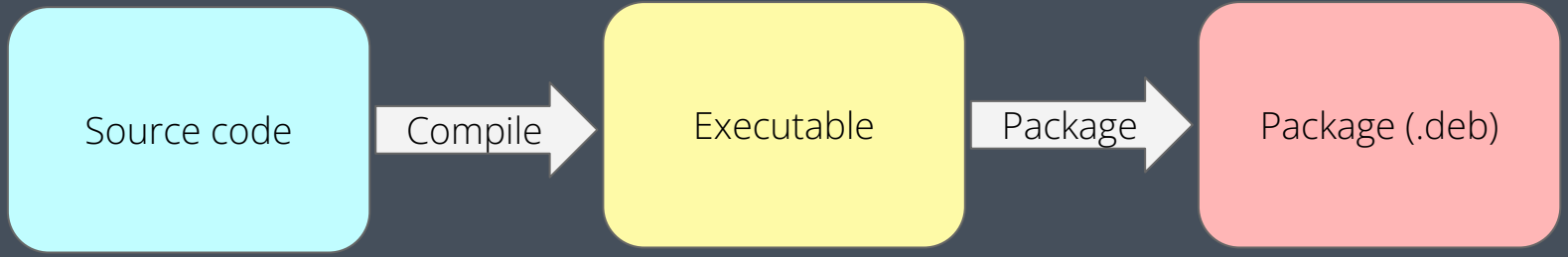
- apt install:
  - Reads from the package lists
  - Finds out what **dependencies** the package needs
  - Checks what packages are already installed
  - **Download** the packages (if not installing a local .deb)
  - **Verify** integrity of packages
  - **Unpacks** them and copies the files over
  - Processes any remaining triggers
    - Triggers are events such as scripts that run post-install
      An example is starting the application as a service

Specifying repositories with sources.list

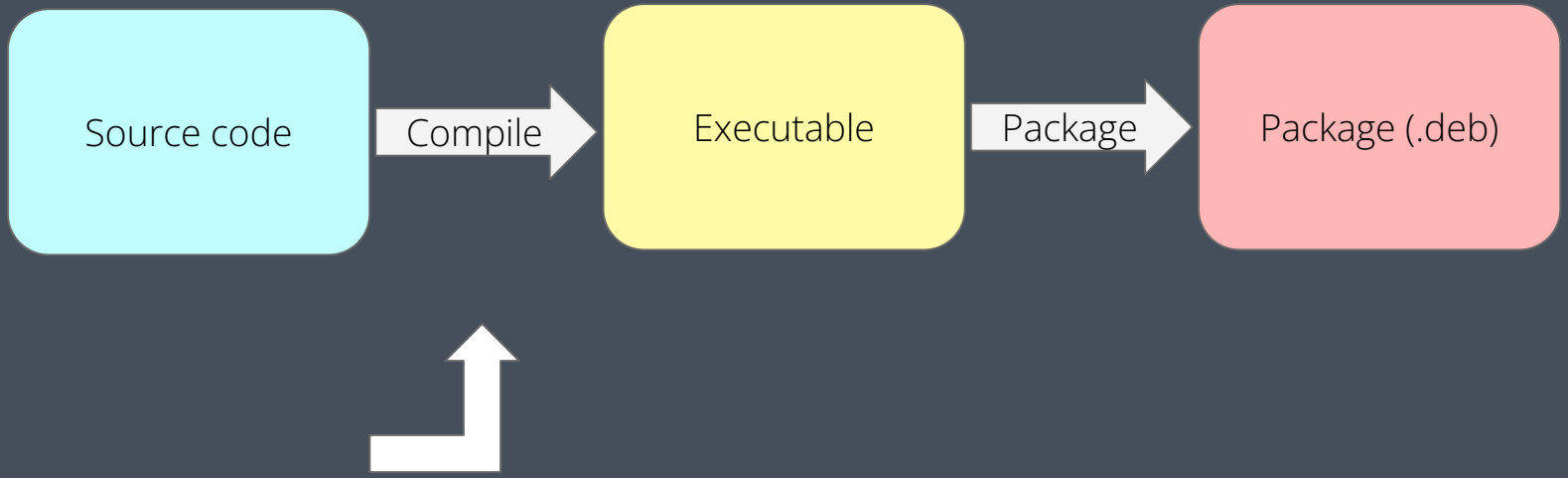
- How does Debian know which package to use?
  - /etc/apt/sources.list and /etc/apt/sources.list.d/*
  - Check on your decalvm!

Creating Packages

Package Creation 101



Package Creation 101



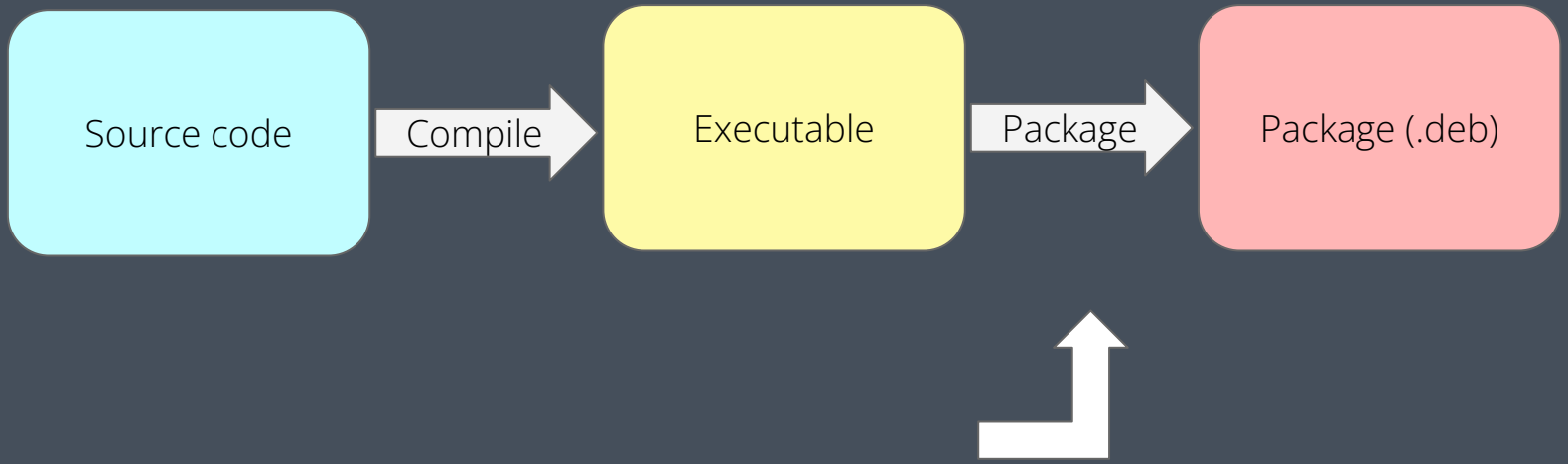
What is compilation?

- Turns source code into an executable binary
- Simplest: gcc hellopenguin.c -o hellopenguin
- Becomes complicated for large projects

Manually Compiling Software

- Compiling software is complicated
- Different tools exist
  - Make, autotools, CMake, etc.
- Follow instructions provided by project!
  - Often, `./configure`, `make`, `make install`

Package Creation 101



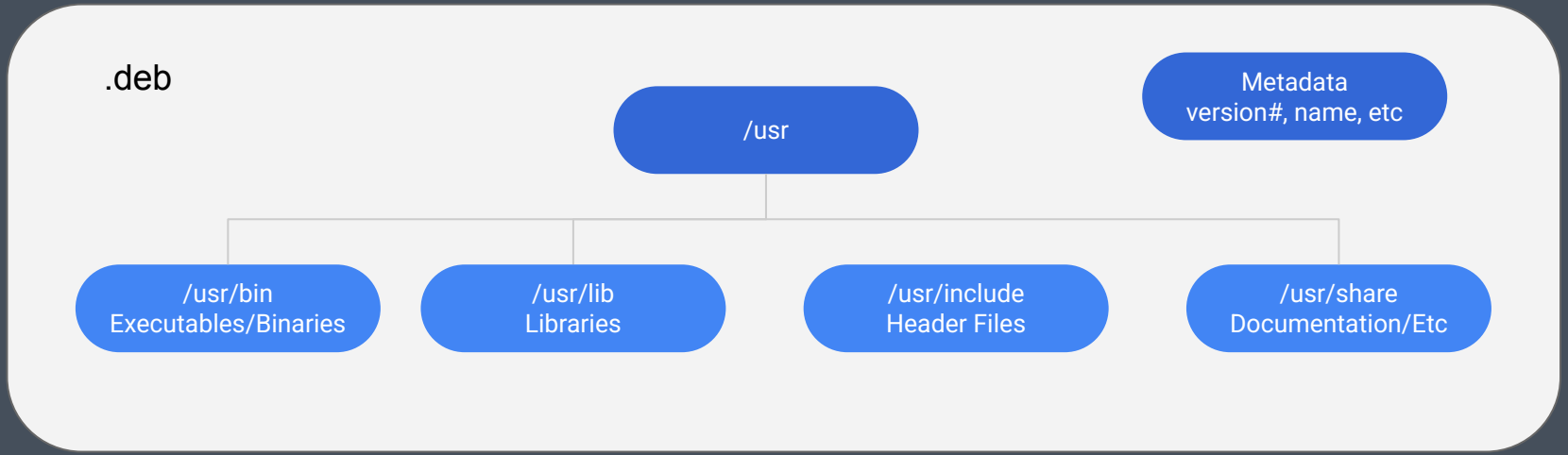
Ok now, how do I make a package?

- Huge pain in the butt
  - Download software/write it
  - Make sure everything configured
  - List dependencies, metadata (version, author, etc.)
  - Make sure linked libraries are in place
  - Use tool to make package

Packaging

- In the lab we're going to use Effing Package Management (FPM), which is a Ruby Gem and makes packaging (slightly) less painful
- `$ sudo apt install ruby-dev`
- `$ sudo gem install fpm`
- `$ fpm -s dir -t deb -n [name here] -v [version #] -C [the directory with the /usr folder]`

Packaging Basics - Package layout



Anatomy of a Debian package



Demo: Spelunking in a .deb

Spelunking in a .deb

- Downloading a .deb w/ `apt download wget`
- Breaking it open with `ar`
- Examining the control file
- Examining the data file

Anatomy of a Debian Package

- How can i view a package deb?
- .deb files are just ar archives
- apt download cowsay && ar x cowsay*_amd64.deb

Anatomy of a Debian Package

- control: Package metadata
- Size of package
- Package version
  - For package updates
- Dependencies of this package

Anatomy of a Debian Package

- /usr/bin/: Executable(s) the package provides
- Added to your $PATH
- /usr/share:
  - Documentation
  - Manpages
  - locales
- /etc: global configuration files
- md5sums: verify integrity of all files

Resources

- Reading List (accessible under decal.ocf.io/resources)

FPM documentation

- https://github.com/jordansissel/fpm/wiki

Debian documentation

- https://www.debian.org/doc/